ASTR 3070: Data Reduction Assignment

Due Date: Novemeber 5th, 2026 at 17:00

1 Part 1: Data Reduction Pipline

This section covers stacking and image calibration. Please read the entire file before getting started.

1.1 Pipeline Setup

1.1.1 Import Libraries

This assignment can be completed without the use of additional libraries. For package issues consult the README.md file.

[ ]:
```
# given libraries
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
from astropy.io import fits

# helper functions - See provided API document
from helper_functions import *
```

1.1.2 Load References to Files

We want to load the filenames of our data into lists for easy reference. This should include: - Lights - Darks - Bias - Flats

[ ]:
```
# define the paths to each folder here
darks_directory = "./NGC 7089/DARK"     # fill in the rest yourself

# fetch filenames for each directory -> list of files
darks_files = fetch_filenames(darks_directory)      # fill in the rest yourself
```

1.2 Create Calibration Frames

Create master stacks for Darks, Bias, and Flats

Hint: use the stack_master() helper function, this returns a 2D array (an image)

[ ]:
```
# Create master dark

# Create master bias
```

```
# Crate master flat
```

1.2.1 Apply Corrections and Normalize

Create bias-corrected and normalized flat frames for each set of lights. Recall that corrections here are just the subtraction of one type of calibration frame from another, and that normalization here means dividing by the maximum value in the array.

Hint: np.max() may be useful here

[ ]:
```
# normalized flat frames
```

1.3 Forming the Image

1.3.1 Calibrate Light Frames

In this step, we want to correct our light frames using our darks and flat/bias correction frames we made earlier. We will obtain a sequence of calibrated light frames for each night. This step may take a moment to run.

Hint: use calibrate_lights() with the correct parameters, as found in the helper docs

Tip: use the pattern 'night_#_X' for your variable names, where # is the night and **X** is the filter used

[ ]:
```
# create lists of calibrated light frames for each filter over each night
```

1.3.2 Select Reference Frame

Select a good quality image from your light frames in ONLY ONE filter (ex: R) to use as a reference frame. All your filter stacks should be aligned to the same reference image, that way the pixel coordinates match for future steps.

Hint: use night_#_X[index] to open the file at position 'index' in the list (index starts at 0)

[ ]:
```
# store reference frames as variables
```

1.3.3 Align Images and Stack

Create a master stack for each type of filter used. This step may take a while to run depending on the number of frames.

Hint: use align_images() and pass all the nights you created above as a single list/tuple, ex: (night_1, night_2, ...)

Tip: Python allows you to store data returned by function in multiple varaibles, ex: a, b = function()

[ ]:
```
# create master images
```

1.3.4 Adjust Header With Correct Exposure Time

An HDU (Header Data Unit) is the structure of a FITS file. As the name suggests, it contains both a header and the data itself. We can take advantage of this and store information about our image in the header portion for quick access. We want to calculate the total integration time for each filter and store this in the image header of an HDU.

Hint: use fits.open("light_filename")[0] to open a specific light FITS file, then access its header (FITS are typically lists of multiple HDU objects, index 0 just gives the image itself)

Tip: it might be helpful to print out the header so you can inspect its contents

Tip: use header['____'] and fill it in with the key you want to access

[ ]:
```
# open HDUs and store a header in a variable for each type of filter used


# time for each subexposure (may need to be changed)
EXPOSURE_TIME = 120

# update header variables - set the exposure to (number_of_frames * subexposure)


# print updated headers to verify
```

1.3.5 Save the Aligned Images in HDU Format and Write to File

We want to combine our master light stacks and the headers we just made into a new HDU object and then write it to a file.

Hint: use fits.PrimaryHDU(image, header) to create a new HDU object

Hint: use .writeto("new_file_name.fits", overwrite=True)

[ ]:
```
# create a new HDU for each filter


# write to file
```

1.4 Plot Final Images

Plot a preview of the master stack (duplicate this block to plot a different filter).

Hint: open the new file you just created using fits.open("filename")

Hint: remember that the HDU is both a header and an image, use image[0].data to get the array and image[0].header if you want the header

[ ]:
```
# open image and retrieve the data
image_data = None

# plot image (repeat for each filter)
```

```
plt.imshow(image_data, cmap='gray', norm=LogNorm(vmin=1, vmax=100000))
plt.title("Reduced [_] Band Image")
plt.colorbar()
plt.show()
```